Guide d'étude : introduction aux réseaux en biologie

Ce qu'il faut savoir faire pour les oraux et l'évaluation sur table

Ce que ce module ajoute

Les modules précédents produisaient des **listes** : des gènes différentiellement exprimés, puis des fonctions enrichies. Ce module s'intéresse aux **relations** entre les éléments : qui interagit avec qui, qui varie avec qui. On apprend à représenter un système comme un **graphe** (des nœuds, des liens), à le construire dans R avec igraph, à le décrire par quelques nombres (degré, distance, intermédiarité, *clustering*) et à y chercher des **modules**. Les exemples vont d'un réseau social (Game of Thrones) à un réseau de co-expression calculé sur `airway`.

Comme aux modules précédents, les erreurs sont presque toutes **silencieuses** : un graphe **dirigé** sans le savoir (`graph_from_data_frame()` l'est par défaut), des gènes **isolés** qui disparaissent du réseau, un attribut `weight` lu par igraph comme une **distance** alors qu'il mesure une proximité, une figure dont la disposition change à chaque exécution, un **seuil** de corrélation arbitraire, des corrélations fortes **par hasard** avec 8 échantillons, `abs(cor)` qui mélange gènes induits et réprimés, des modules de Louvain qui changent avec la graine, un « hub » qui n'est qu'un gène très étudié, un module qui n'est que le **sexe du donneur**. Aucune n'arrête R, et un assistant d'IA ne peut pas les éviter à votre place : il ne sait pas ce que signifient vos liens, ni ce que mesure votre poids, ni comment vos échantillons ont été choisis.

Comment ce module est évalué

Comme le reste du cours, **en présence et sans outil d'IA** :

- à l'**évaluation sur table** (50 % de la note du cours) : [À PRÉCISER : sur papier ou en salle informatique surveillée, durée, documents permis] ;
- lors des **oraux** individuels (25 % chacun) : [À PRÉCISER : quel oral porte sur ce module].

On ne vous demandera pas de retenir la liste des fonctions d'igraph ni les dates des articles. On vérifiera que vous savez **lire** un objet igraph et une sortie (`degree()`, `distances()`, `betweenness()`, une table de modules), **calculer** à la main un degré, une densité, une distance ou un *clustering* sur un petit graphe, **prédire** l'effet d'un réglage, et **repérer** une conclusion fausse.

Les types de questions

Type	Ce qu'on vous demande	Exemple déjà vu
Prédire	Donner le résultat sans exécuter	L'en-tête et le degré de `TF2` dans un petit graphe dirigé (diapos « À vous : lire un objet igraph »)
Erreur	Expliquer pourquoi un résultat est faux	Le « régulateur central » trouvé par `betweenness()` (diapos « À vous : lire une analyse de réseau »)
Et si	Anticiper l'effet d'une modification	Passer le seuil de corrélation de 0.8 à 0.7 ou 0.9 (diapos, TP2)
Expliquer	Justifier un concept avec vos mots	Pourquoi les ponts de Königsberg sont infranchissables (diapos)
Lire	Interpréter une sortie de R	L'en-tête `UNW- 127 550`, une table module × statut (diapos, TP1, TP2)
Calculer	Un descripteur, à la main	Degré, densité, distance, *clustering* local sur un graphe de cinq nœuds (diapos « À vous : prédire des descripteurs »)
Choisir	Choisir une représentation ou une méthode	Dirigé ou non, pondéré ou non, pour un réseau biologique (diapos « À vous : trouver un réseau »)

À l'oral, chaque réponse est suivie d'une **relance** : « pourquoi ? », « comment l'auriez-vous vérifié ? ».

Ce que vous devez savoir faire

G-GRA : les graphes

- Utiliser le vocabulaire : **nœud** (sommet, *vertex*), **lien** (arête, *edge*), N, L, **voisin**, **degré**.
- Énoncer la condition d'Euler : un chemin passant une seule fois par chaque lien existe dans un graphe connexe si et seulement si **zéro ou deux** nœuds ont un degré impair ; l'appliquer à Königsberg et à une figure.
- Distinguer graphe **dirigé** et **non dirigé**, **pondéré**, avec **boucles**, **multigraphe**, **bipartite** ; dire ce que perd une **projection** d'un graphe bipartite.
- Expliquer pourquoi le choix de la **représentation** change la difficulté du problème (graphe de chevauchement et chemin hamiltonien contre graphe de de Bruijn et chemin eulérien).

Où le travailler : section 2 des diapos (pauses « À vous : sans lever le crayon » et « À vous : trouver un réseau »), exercice formatif Q1.

G-REP : représenter un réseau dans R

- Passer d'une **liste d'arêtes** à une **matrice d'adjacence** et inversement ; savoir qu'elle est symétrique si le graphe est non dirigé, que la somme d'une ligne est le **degré**, et qu'un graphe dirigé se lit « ligne → colonne ».
- Expliquer pourquoi on stocke une liste d'arêtes plutôt qu'une matrice (les réseaux réels sont **creux**) ; calculer une **densité**.
- Construire un graphe avec `graph_from_data_frame()` (**dirigé par défaut** : `directed = FALSE`) et `graph_from_adjacency_matrix()` ; savoir ce que fait l'argument `vertices`.
- **Lire l'en-tête** d'un objet igraph : `U` / `D`, `N`, `W`, `B`, le nombre de nœuds et de liens, les attributs `(v/c)`, `(e/n)`.
- Ajouter et lire des **attributs** (`V(g)$sex`, `E(g)$weight`) en respectant l'**ordre de `V(g)`**.
- Prédire la perte des nœuds **isolés** quand on construit un graphe à partir d'une liste d'arêtes seule.
- Dessiner un graphe et savoir qu'une **disposition** est un calcul souvent aléatoire : la position d'un nœud à l'écran n'est pas une donnée.

Où le travailler : section 3 des diapos (pause « À vous : lire un objet igraph »), TP1 (construire le graphe Game of Thrones, attributs et sexe, dessin et dispositions, matrice d'adjacence, degré par `count()`), exercice formatif Q2.

G-BIO : les réseaux en biologie

- Citer les principaux réseaux biologiques (interactions protéiques, régulation, métabolique, co-expression, maladies) : leurs nœuds, leurs liens, dirigés ou non.
- Dire d'où viennent les liens : **mesurés** (double-hybride, co-IP, ChIP-seq), **prédits** (contexte génomique, fouille de texte, score STRING) ou **calculés** à partir de vos données (co-expression).
- Construire un réseau de co-expression : matrice d'expression, corrélation par paire, **seuil**, graphe ; le faire à la main sur une petite matrice de corrélations.
- Expliquer pourquoi, avec **8 échantillons**, de fortes corrélations apparaissent par hasard.
- Expliquer le problème de l'**inférence** : une corrélation n'est ni une interaction ni une régulation ; une corrélation **marginale** peut venir d'un lien **indirect**.

Où le travailler : section 4 des diapos (« De l'expression au graphe », « Piège : 8 échantillons »), TP2, partie B (co-expression `airway`), exercice formatif Q5.

G-TOP : les propriétés topologiques

- Calculer à la main un **degré**, un degré entrant et sortant, une **force** (somme des poids), une **densité**.
- Calculer le degré à partir d'une liste d'arêtes avec `count()` (les **deux** extrémités de chaque lien).
- Lire une **distribution des degrés** ; distinguer réseau aléatoire et réseau à **queue lourde** ; définir un **hub** et la règle centralité-létalité ; dire pourquoi un réseau invariant d'échelle résiste aux pannes au hasard mais pas aux attaques ciblées.
- Calculer une **distance**, un **diamètre**, identifier le nœud de plus forte **intermédiarité** sur un petit graphe ; savoir qu'une distance est **infinie** entre deux composantes.
- Calculer un **coefficient de *clustering*** local, $C_i = 2\,e_i / [k_i\,(k_i - 1)]$, et le comparer à la densité.
- Expliquer le **petit monde** : courtes distances et fort *clustering*.
- Trouver les **composantes** d'un graphe et savoir que `mean_distance()` ignore les paires non reliées.

- Prédire l'effet d'un attribut `weight` sur `distances()`, `shortest_paths()`, `diameter()` et `betweenness()` ; choisir `weights = NA` ou `1 / weight`.

Où le travailler : section 5 des diapos (pause « À vous : prédire des descripteurs », « Piège : les poids sont des distances »), TP2, partie A (degrés, intermédiarité avec et sans poids, chemin de Jon à Daenerys, voisinage), TP2, partie B (isolés, composantes, hubs), exercice formatif Q3 et Q4.

G-MOD : modules et communautés

- Définir une **clique** et expliquer pourquoi c'est une notion trop stricte pour trouver des modules.
- Définir un **module** et la **modularité** Q (liens internes observés moins attendus au hasard) ; citer une méthode agglomérative (Louvain), divisive (Girvan-Newman), par marches aléatoires (walktrap).
- Lancer `cluster_louvain()`, lire `membership()`, `sizes()`, `modularity()`.
- Croiser les modules avec une **annotation** (statut DE, chromosome) et **interpréter** : un module dit que des gènes varient ensemble, pas pourquoi.
- Savoir que Louvain est **aléatoire** (graine, stabilité) et que les numéros de modules sont des **étiquettes**.

Où le travailler : section 6 des diapos (« Les modules de co-expression », « Un module… le sexe du donneur »), TP2, partie A (cliques, Louvain sur Game of Thrones), TP2, partie B (modules × statut × chromosome, module du Y), exercice formatif Q6.

G-AUD : vérifier une analyse de réseau

Devant une analyse de réseau, demander :

- le graphe est-il **dirigé** ? (lire `U` / `D` dans l'en-tête) ;
- combien de **nœuds** avant et après la construction ? des gènes **isolés** ont-ils disparu ?
- que signifie le **poids**, et igraph l'utilise-t-il comme une **distance** ?
- la figure est-elle **reproductible** (`set.seed()`, disposition enregistrée) ? les conclusions viennent-elles des **calculs** ou du dessin ?
- combien d'**échantillons** ? quel **seuil** ? la conclusion résiste-t-elle à un autre seuil ?
- corrélations **signées** ou `abs()` ?
- les modules sont-ils **stables** d'une graine à l'autre ? que reflètent-ils (traitement, donneur, sexe, lot) ?
- un **hub** est-il un régulateur, ou un gène très **étudié** ?
- `components()` renvoie une erreur `no applicable method` : la fonction d'igraph est **masquée** par un package chargé après lui (`igraph::components()`).

Où le travailler : les diapos « Piège » des sections 3 à 6, la pause « À vous : lire une analyse de réseau », les deux TP, exercice formatif Q7.

Comment vous préparer

1. **Refaites sans ordinateur** les pauses « À vous » et l'exercice formatif : écrivez votre prédiction, *puis* vérifiez dans RStudio.

2. Après chaque construction de graphe, lisez la **première ligne** : `U` ou `D` ? combien de nœuds, combien de liens ? Est-ce ce que vous attendiez ?
3. Entraînez-vous aux **calculs à la main** sur de petits graphes de 5 ou 6 nœuds : degré, densité, distance, diamètre, *clustering* local, matrice d'adjacence. Vérifiez avec `degree()`, `edge_density()`, `distances()`, `transitivity(type = "local")`.
4. Devant chaque résultat, demandez-vous ce qu'il deviendrait **sans les poids**, **avec un autre seuil**, **avec une autre graine**.
5. Les solutionnaires des TP donnent les réponses et les explications : lisez surtout les explications.